DATA ENGINEERING

Change Data Capture Visual Guide

Stream every insert, update, and delete.
Log-based CDC with Debezium and the outbox pattern.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Why Capture Changes?

Batch ETL re-reads whole tables. CDC streams only what changed.

Batch

SELECT * nightly — 10M rows

CDC

stream 50k changed rows

CDC gives near-real-time freshness at a fraction of the source load — and captures deletes, which batch snapshots miss entirely.

STEP 2/6

Three CDC Approaches

Polling, triggers, and log-based — only one scales cleanly.

Polling (timestamps)

WHERE updated_at > last_run. Misses deletes; late-arriving rows slip.

Triggers

DB trigger writes to a shadow table. Correct but slows every write.

Log-based

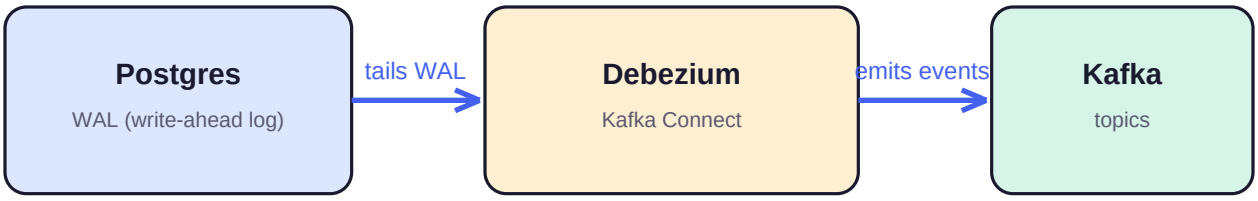
Read the DB's replication log. No impact on writes, captures everything.

Log-based CDC piggybacks on the mechanism databases already use for replication — it's the only approach with near-zero source overhead.

STEP 3/6

Debezium: Log-Based CDC in Practice

A Kafka Connect source connector that tails the database log.



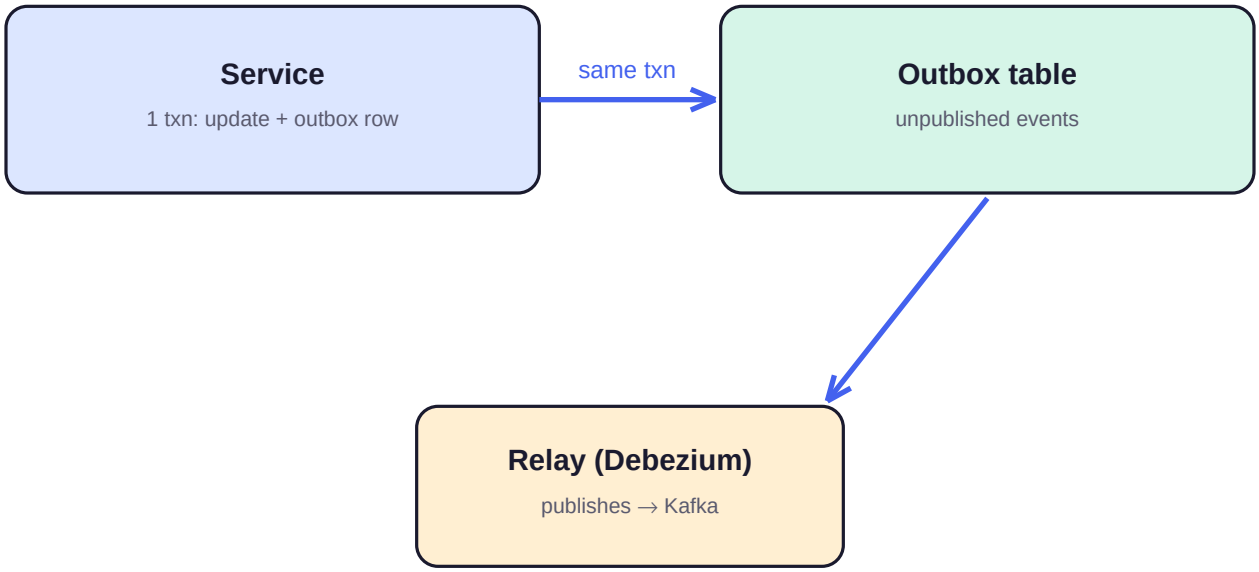

```
{"op": "u", "before": {...}, "after": {...}}
op: c=create, u=update, d=delete, r=snapshot read
```

Each event carries before/after images and the source position. One topic per table; consumers replay from any offset for reprocessing.

STEP 4/6

The Outbox Pattern

Atomically publish domain events from a transactional service.



The dual-write problem: updating the DB and publishing to Kafka can't both be atomic — unless the 'publish' is a row in the same transaction. The relay does the actual publishing.

- Outbox rows are deleted (or marked) after publishing
- Consumers must be idempotent — at-least-once delivery
- The pattern works for any service that owns a database

STEP 5/6

Ordering & Delivery Guarantees

What CDC promises — and what it doesn't.

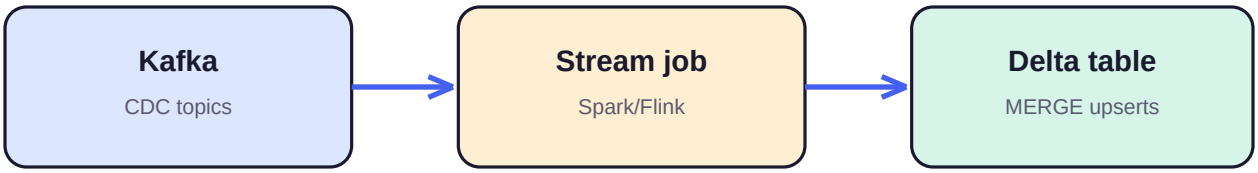
- Per-row ordering: events for one row arrive in commit order (same topic partition)
- Cross-table ordering: NOT guaranteed — join carefully downstream
- Snapshots: initial consistent snapshot before streaming (no missed rows)
- At-least-once delivery: sinks must dedupe on (table, key, position)
- Schema changes in source propagate — pin consumer schemas deliberately

Design sinks for idempotent upserts keyed on the source primary key — replays become safe instead of scary.

STEP 6/6

Landing CDC into the Lakehouse

From Kafka topics to queryable tables.



The canonical landing: streaming MERGE INTO the Delta table on primary key, handling c/u/d ops. Deletes become tombstones; late data replays cleanly.

- Bronze: raw CDC events, append-only (audit trail)
- Silver: current state via MERGE (queryable)
- Monitor lag: source position vs consumer offset per table

STEP ✓/✓

Key Takeaways

Change Data Capture — Remember

- CDC streams only changed rows — fresher than batch, lighter on the source, and it captures deletes.
- Log-based CDC (Debezium tailing the WAL) has near-zero write impact, unlike polling or triggers.
- The outbox pattern makes event publishing atomic with the business transaction.
- Expect at-least-once delivery and per-row (not cross-table) ordering — sinks must be idempotent.
- Land with streaming MERGE: bronze keeps the raw event history, silver holds current state.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev